Level 1 — Micro-CMS v1

Vulnerability Report

Program: Hacker101 CTF
Target: Level 1 — Micro-CMS v1
Findings: 4 confirmed

Executive Summary

Micro-CMS v1 is a small content-management style application that allows users to create, view, and edit pages through form submissions. Assessment identified **four confirmed vulnerabilities** spanning stored cross-site scripting (two distinct sinks), broken access control on a private page, and SQL injection via a resource identifier.

Taken together, the findings show inconsistent trust boundaries:

- Input filtering on page **body** content was stricter than filtering on **title** and **event-handler attributes**.
- Authorization on the **view** path for a private page was not mirrored on the related **edit** path.
- A resource identifier used in routing/queries was not safely parameterized.

In a production CMS, this combination enables session theft / account takeover via XSS, unauthorized disclosure or modification of private content via access-control gaps, and database compromise via injection. Each finding below includes severity, impact, and concrete remediation guidance.

Detailed offensive walkthrough notes are retained in `CTFs/Raw_Notes/`. Proof-of-concept sections in this report are validation summaries with flag evidence, not reproduction cookbooks.

Finding 1: Stored Cross-Site Scripting (XSS) in Page Title

Field	Value
Severity	High
Vulnerability Class	Stored Cross-Site Scripting
CWE	CWE-79 (Improper Neutralization of Input During Web Page Generation)
Related Flag	Flag 2

Description

The application applies script-scrubbing controls to page **body** content (markdown is supported; script constructs are replaced or neutralized). The **title** field does not receive equivalent protection.

Crafted markup stored in the title is persisted to the backend and later rendered into pages that list or display that title (notably the homepage after navigation/re-render). Because the stored title is reflected into an HTML context without robust output encoding, active content executes in the victim's browser under the application origin.

This is a classic "filter on one field, forget another" defect: defending the rich-text body is insufficient when titles, metadata, or alternate sinks remain trusted as plain text but are rendered as HTML.

Proof of Concept

Validation summary (non-actionable):

1. Page edit functionality was used to persist active markup in the **title** field. The value was stored without the scrubbing behavior observed on the body field.
2. Immediate post-save rendering did not necessarily trigger execution; returning to the homepage forced a re-render of stored titles.
3. On homepage re-render, the stored title executed in the browser context, confirming stored XSS.
4. Successful exploitation in the CTF environment was evidenced by:

```
^FLAG^7c27cf87146d039590fd323c36e41f69913e4fd9ef79149e2aff366c4c26d7c1$FLAG$
```

Impact

- **Session compromise:** Stolen cookies/tokens can enable account takeover if session identifiers are accessible to script.
- **Persistent wormability:** Any user (or admin) who views an affected listing page becomes a victim.
- **UI redress / phishing:** Injected content can rewrite trusted UI under the legitimate origin.
- **Defense bypass lesson:** Body-only scrubbing creates a false sense of security while titles remain a high-value sink.

Mitigation

1. **Output-encode by context.** Treat titles as untrusted data. When inserting into HTML text nodes, use HTML entity encoding; never concatenate raw user input into markup.
2. **Do not rely on input scrubbing alone.** Allowlist sanitizers (if rich HTML is required) must run on **every** field that can reach an HTML sink, including titles, alt text, and button labels — or, preferably, store titles as plain text and encode on output.
3. **Use a templating engine with auto-escaping enabled** by default; audit any `|safe` / `raw` / disabled-escape paths.
4. **Deploy a Content Security Policy (CSP)** that disallows inline scripts (`script-src` without `'unsafe-inline'`) as defense-in-depth.
5. **Set cookies `HttpOnly` (and `Secure` / `SameSite` appropriately)** so successful XSS cannot trivially exfiltrate session cookies.
6. **Regression tests:** attempt to store active markup in title, body, and metadata fields; assert encoded output and absence of executable nodes.

Affected Assets

- Page create/edit form — `title` field
- Homepage / page listing views that render stored titles
- Any other template that echoes page titles into HTML without encoding

Finding 2: Broken Access Control (IDOR) on Private Page Edit Path

Field	Value
Severity	High
Vulnerability Class	Broken Access Control / Insecure Direct Object Reference
CWE	CWE-639 (Authorization Bypass Through User-Controlled Key), CWE-284
Related Flag	Flag 0

Description

Pages are addressed by numeric identifiers (e.g., view vs edit routes for a given page ID). During identifier enumeration, most unknown IDs returned a not-found style response, while one ID returned a **Forbidden** response on the view path — indicating the object exists but the current principal is not allowed to read it.

The related **edit** route for that same object did not enforce the same authorization decision. Navigating to the edit interface for the private page exposed its contents (titled “Private Page”), including sensitive data, despite the view path denying access.

This is a textbook broken-access-control pattern: authorization is implemented inconsistently across operations on the same object. Existence oracles (Forbidden vs Not Found) further assist attackers in locating protected IDs.

Proof of Concept

Validation summary (non-actionable):

1. Sequential probing of page identifiers showed differentiated responses: generic not-found for missing objects versus Forbidden for at least one existing private object on the view path.
2. The edit interface for that same private page ID was reachable without the authorization barrier applied to viewing.
3. The edit form disclosed private page content, confirming unauthorized read access via the weaker endpoint.
4. Confirmed with flag evidence:

```
^FLAG^f39663c6cdca64a5749f5f5f9b7719d696b2b58572888b5f159155765a32f51c$FLAG$
```

Impact

- **Unauthorized disclosure** of private or draft content.
- **Potential unauthorized modification** if edit/save is also permitted without checks (even read-only exposure of an edit form is a serious confidentiality failure).
- **Object enumeration:** Distinct Forbidden vs Not Found responses help attackers map sensitive resources.
- In production, this class of bug frequently exposes other users' documents, tickets, invoices, or admin-only pages.

Mitigation

1. **Centralize authorization.** For every operation (view, edit, delete, export), load the object then enforce the same policy (ownership, role, `public` / `private` flag) before returning data or a form.
2. **Deny by default.** If the principal cannot view an object, they must not receive edit UI, API JSON, or pre-filled fields for that object.
3. **Normalize failure responses** where appropriate (uniform 404 for unauthorized and missing) if confirming object existence is itself sensitive.
4. **Server-side checks only.** Never rely on hiding links in the UI; direct URL access must be authorized.
5. **Automated tests:** for a private page, assert that anonymous and unauthorized users receive denial on *both* view and edit (and on POST save).
6. **Audit IDOR surface:** any route containing `/page/{id}`, `/edit/{id}`, or similar must share one authorization helper.

Affected Assets

- Page view routes keyed by page ID (authorization present for private page)
- Page edit routes keyed by the same page ID (authorization missing / weaker)
- Private page content ("Private Page")

Finding 3: Stored XSS via HTML Event Handler (Filter Bypass)

Field	Value
Severity	High
Vulnerability Class	Stored Cross-Site Scripting (Filter Bypass)
CWE	CWE-79, CWE-184 (Incomplete Filtering of Special Elements)
Related Flag	Flag 3

Description

Page body editing advertises that markdown is supported but scripts are not. Attempts to store conventional script elements in the body are neutralized (observed as scrubbed placeholders in rendered output).

However, the sanitizer is incomplete. HTML that introduces an interactive element with an **event-handler attribute** (for example, a button with a client-side click handler) was accepted, stored, and later rendered. Activating the control executed attacker-controlled script in the application origin.

This demonstrates why deny-list scrubbing of `<script>` tags is insufficient: the HTML attack surface includes event handlers, SVG/math vectors, `javascript:` URLs, and other sinks. Robust defense requires allowlisting and/or strict contextual encoding — not pattern removal of one tag.

CTF note: In this lab, successful script execution was directly tied to flag revelation. In production, impact is typically session theft, actions as the victim, or malware delivery — not an explicit “flag” callback.

Proof of Concept

Validation summary (non-actionable):

1. Body content containing a conventional script element was scrubbed/neutralized as expected.
2. Body content that embedded an HTML control with an event-handler attribute was stored without equivalent neutralization.
3. Interacting with the rendered control caused script execution in the page origin; page source confirmed the handler remained intact.
4. Confirmed with flag evidence:

```
^FLAG^562412504ddf8e2d1f5f3f1e5bb210ef101c37a4185cfd763c5fec5e0861f55a$FLAG$
```

Impact

- Same practical impact class as Finding 1 (session theft, persistent XSS, UI compromise), with the added concern that **developers may believe XSS is "already fixed"** because `<script>` scrubbing is visible.
- Attackers preferentially seek filter gaps; incomplete sanitizers often create a false sense of security in code review and QA.

Mitigation

1. **Replace deny-list scrubbing with an allowlist HTML sanitizer** (well-maintained library configuration) that strips all event-handler attributes (`on*`), `javascript:` URLs, and dangerous tags/elements by default.
2. If markdown is the intended authoring format, **render markdown to a safe HTML subset** through a pipeline that never passes raw attacker HTML through unchecked.
3. **Prefer plain-text storage + encode-on-output** when rich HTML is unnecessary.
4. **CSP without** `'unsafe-inline'` and without allowing attacker-controlled script sources.
5. **Security regression suite** covering event handlers, SVG, markdown code fences, and nested tags — not only `<script>` .
6. Align title and body defenses (see Finding 1) so one sink cannot remain weaker than another.

Affected Assets

- Page body editor (markdown / HTML input)
- Rendered page views that emit stored body HTML (including sample content with interactive elements)

Finding 4: SQL Injection via Page Identifier

Field	Value
Severity	Critical
Vulnerability Class	SQL Injection
CWE	CWE-89 (Improper Neutralization of Special Elements used in an SQL Command)
Related Flag	Flag 1

Description

Page resources are selected using an identifier supplied through the URL. That identifier is incorporated into a backend database query without adequate parameterization or type-safe handling.

When the identifier is manipulated to include SQL metacharacters, the application's database layer interprets attacker-controlled input as part of the query structure rather than as a bound value. In the CTF environment, this behavior was sufficient to confirm injection and retrieve the associated flag.

Even "simple" ID parameters are a common injection point when frameworks concatenate path segments into SQL, especially in older or hand-rolled CMS routing.

Proof of Concept

Validation summary (non-actionable):

1. Page-view requests that supply a normal numeric identifier behave as expected.
2. Supplying a page identifier containing SQL metacharacter content altered backend query handling in a way inconsistent with safe parameterized lookup (error/behavior differential confirming injection).
3. The injection condition was accepted by the challenge as a successful finding, evidenced by:

```
^FLAG^e158136e2ac697fca06fb91fdffedf2f0d3424ac1a289beea34d3d59c270388f$FLAG$
```

Impact

- **Confidentiality:** read arbitrary data the DB role can select (users, content, secrets).

- **Integrity / availability:** modify or destroy data; in some engines, pivot to OS command execution depending on configuration.
- **Full application compromise** when combined with weak DB privileges or stacked queries.
- Severity is Critical because injection on a routinely requested identifier is typically trivial to discover and highly leveraged.

Mitigation

1. **Use parameterized queries / prepared statements exclusively** for all SQL. Bind the page ID as a typed parameter (integer), never string-concatenate it into SQL.
2. **Validate and coerce types early** (reject non-integer IDs at the routing layer) as defense-in-depth — not as the primary control.
3. **Least-privilege database accounts** for the web app (no DDL, no unnecessary file/OS privileges).
4. **ORM / query builders** correctly used (avoid raw SQL string interpolation helpers).
5. **Error handling:** do not return raw database errors to clients; log server-side.
6. **SAST/DAST and code review** for any `execute("... " + userInput)` patterns; add a regression test that asserts metacharacters in IDs cannot change query structure.

Affected Assets

- Page view (and potentially edit) routes that accept a page ID from the URL
- Backend query path that loads page rows by ID

Appendix: Approaches Tried

The following activities were part of reconnaissance or attempted exploit paths. They did not each produce a flag on their own, but they clarify filter boundaries and reduce false assumptions about the attack surface.

Approach	Result / Learning
Inspecting `document.cookie` in the browser console	Session-like values present; not themselves flags; retained as potential XSS impact targets
Requesting `/favicon.ico` by path append (pattern from Level 0)	Standard 404; not a disclosure vector here
Enumerating early page IDs (`page/3` , `page/4` , etc.)	Many IDs absent; later enumeration revealed Forbidden vs Not Found differential (led to Finding 2)
Storing `<script>` in page body / markdown	Scrubbed / neutralized — body script tags are defended
Markdown code-block / nested script attempts in body	Did not yield execution via those avenues
Observing lack of CSRF token on edit forms	Notable hardening gap; not pursued as a standalone confirmed flag finding in these notes
Title-field stored XSS	**Confirmed** (Finding 1)
Private page edit-path access	**Confirmed** (Finding 2)
Event-handler attribute in body HTML	**Confirmed** (Finding 3) — demonstrates incomplete scrubbing
SQL metacharacter in page ID	**Confirmed** (Finding 4)

Takeaway for defenders: Partial XSS filters and per-route authorization create uneven trust. Successful findings clustered where one field, one attribute class, or one HTTP operation was left outside the control applied elsewhere.